

Y COMBINATOR TALKS · COMBINED

# How to build an AI-native SaaS company

A sequential roadmap: what to do before what.

Idea → customer depth → tiny MVP → AI build loop → launch and learn → company as a closed loop.

---

Sourced from four YC partner talks: picking and committing to one idea (John), shipping a real MVP (Twitch / Airbnb / Stripe examples), vibe-coding so you can ship that MVP fast (Tom), and running the company as an AI operating system (Diana).

**The order is the point.** Most founders skip ahead — designing the org, polishing the product, or juggling five ideas — before they have one desperate customer using one small thing. Do the phases in order. You can run Phase 3 and 4 in a tight weekly loop. You cannot skip Phase 1 and 2.

## The seven phases

- **01 Commit** — pick one idea, kill the others
- **02 Go deep** — learn the customer's job like you could run it tomorrow
- **03 Cut the MVP** — 2–6 week deadline, cut most features
- **04 Build with AI** — plan file, tests, one slice, git, reset when it thrashes
- **05 Launch & learn** — hair-on-fire users, iterate 3–6 times
- **06 Close the loops** — make the company readable by AI
- **07 Stay tiny** — builders + DRIs + founder in the tools, tokens not headcount

## The sequence at a glance

Read this page first. The rest of the document is the detail for each step. Do not start Phase 6 (company systems) before you have users touching a product.

| Phase                        | Do this                                                                              | Done when                                                                                   | Do not do yet                                                                               |
|------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>01 Commit</b>             | Pick one idea. Kill the others. Become that company.                                 | You can say the idea in one sentence and you are no longer working on anything else.        | Building two products. Waiting for the perfect idea.                                        |
| <b>02 Go deep</b>            | Learn the customer's job so well you could run their business tomorrow.              | You know their daily crises, what they lose when the problem hits, and what they would pay. | 100 surveys instead of conversations. Claiming founder-market fit without talking to users. |
| <b>03 Cut the MVP</b>        | Write a tiny spec. Cut most features. Set a 2–6 week deadline.                       | A desperate customer can use the brick, even if it is ugly.                                 | Year-long builds. Fake-Steve-Jobs perfect products. Fundraising before a product exists.    |
| <b>04 Build with AI</b>      | Plan in markdown. Tests first. One section at a time. Git after every working slice. | Each slice is committed, tested, and reversible. The model is not free-running in a mess.   | Oneshotting the whole app. Stacking failed prompts on dirty code.                           |
| <b>05 Launch &amp; learn</b> | Put it in front of hair-on-fire users. Iterate 3–6 times. Recruit one by one.        | Real usage data. Users who watched it improve. You know if the pain is real.                | Hiding from the first “no.” Chasing 100,000 people who kind of like it.                     |
| <b>06 Close the loops</b>    | Make the company readable by AI. Record work. Agents in tickets, chat, docs, sales.  | Status, decisions, and outcomes feed back into one intelligence layer.                      | Hiring coordinators to pass information. Keeping the company in DMs and memory.             |
| <b>07 Stay tiny</b>          | Three roles: builders, DRIs, AI-native founder. Spend on tokens, not headcount.      | One person + agents replaces a pre-AI team. Founder still uses the tools daily.             | Classic middle-manager org chart. Outsourcing “the AI strategy.”                            |

**How the four talks map onto this.** John = Phases 1–2 and the “if it fails” section. MVP talk = Phases 3 and 5. Tom = Phase 4 (how you physically build). Diana = Phases 4 (software factories), 6 and 7 (company as OS). Use all four. None of them replace the others.

### 01 Pick one idea and burn the other boats

You cannot find the perfect idea by thinking harder. You find it by committing, talking to customers, and shipping. Working on several ideas at once gives you bad data — you never go deep enough to know if something is real.

#### Do this

- **Stop hunting for the perfect idea.** Pick one you are curious about. Start.
- **Ignore the “am I allowed to work on this?” voice.** You do not need a decade in the industry. Curiosity + depth + customers can create founder-market fit fast. Blake Scholl went from ad tech to Boom Supersonic.
- **Commit to one.** Stop the other projects. Tell any users you pivoted. Change the name, email, site, and internal story if you have to. Become unrecognizable as the old company. GovDash pivoted ~5 times; the fifth idea took off only after they fully became that company.

- **Prefer ideas with three AI-era qualities:** (1) sits at the edge of what models can do *today* — barely works now, gets better as models improve; know the bottlenecks; a bottleneck that does not clear might *be* the company. (2) Sells an *outcome*, not “software for X.” Software is getting cheap; value is trust, licenses, regulation, owning the result. Don’t sell tools to insurers — be the insurer. (3) The ambitious version. Modest and huge ideas cost almost the same pain. Aim at rewriting a sector.

#### Do not do this yet

- Juggle three startups “to see which works.”
- Wait until you have 10 years of domain experience.
- Design the org chart, raise a big round, or hire a team before you have picked.

**Done when:** You can say the company in one sentence. The other ideas are dead. You have started talking to the people who have the problem.

## 02 Go deep on the customer before you over-build

Going deep is not 600 interviews and no product. It is becoming one of the most informed people on this problem, while you ship. The test: could you run their business tomorrow? Could you teach a class on this problem?

#### Do this

- Hunt **hair-on-fire** customers first. You are selling a brick, not a hose. Only desperate people will use a brick. Everyone else can wait.
- Learn their daily crises. What they lose when the problem happens. Whether this is a top-five problem. What they would actually pay so it never happens again.
- Talk and ship in a tight loop: understand → put a product in front of them → understand more → ship better. Do not wait for 100 interviews before writing code.
- Remember: customers are experts in the **pain**, not the **solution**. Surveys show pain. They do not tell you what to build. The real conversation starts when a product is in their hands.

#### Do not do this yet

- A year of research with nothing to show.
- Building for “the market” instead of 10 desperate people.
- Asking users to design the product for you.

**Done when:** If someone dropped you into the customer’s business tomorrow, you would know how the day actually works — and you already have at least a few of those people who will try a rough product.

## 03 Define the smallest possible MVP — then cut it again

The MVP is not meant to be good. It is meant to start the conversation. First versions of Airbnb, Twitch, and Stripe were embarrassing by later standards. That is why they learned.

#### How small “small” really is

- **Airbnb v1:** no payments, no map, air mattresses only, only during conferences (the site went up and down by city).
- **Twitch v1:** Justin.tv — one page, one person with a camera on his head, almost no games, rented CDN.

- **Stripe v1:** tiny bank, manual paperwork every night, almost no API. Only useful for startups that needed “take a credit card.” Even Twitch could not use it yet.

### How to ship it in weeks, not months

- **Hard deadline:** 2 weeks, 4 weeks, or 6 weeks. No open-ended “when it’s ready.”
- **Write the spec down** so you stop re-debating features every day.
- **Cut.** For each feature ask: does a desperate customer need this *to start*? Most things wait for v2 / v3 / v4.
- **Do not fall in love with the MVP.** Fall in love with the user. The first product will change a lot.
- Better **100 people who love it** than 100,000 who kind of like it. Recruit those first users one at a time. Do things that do not scale.

### Kill these fears

- **“If they hate v1 I lose them forever.”** Early adopters expect broken software. Pitch: it will not be perfect on day one; stay with me and we will make it work. People who would vanish after one bug were never going to try you.
- **“If the first demo fails the company dies.”** You wake up tomorrow. You email someone else. You come back in a week with a better version. Do not spend a year hiding from a “no.”
- **Fake Steve Jobs:** “I already know the perfect product; I need a year and \$10M.” First iPhone had no App Store, no video, 2G. First iPod click wheel jammed. The product people remember is version 3 or 4.

**Done when:** You have a written, cut spec and a date. A desperate user can complete one core job — even if payments, maps, polish, and half the API are missing.

## 04 Build it with AI — plan, tests, slices, git

This is how you actually produce the MVP (and every version after). Two ideas stacked: Tom’s vibe-coding practice, and Diana’s software factory (humans write spec + tests; agents write the code until tests pass). Treat the model like a junior engineer who needs a spec, tests, a clean history, and a reset when it thrashes.

### Before you touch the repo

- **Tool:** never coded → Replit / Lovable for UI experiments (Lovable gets sloppy on precise backend changes). Written code before → Cursor, Windsurf, or Claude Code.
- **Spend “unreasonable” time in a plain LLM first** locking scope and architecture. Do not let an agent free-run in a repo before you know the goal.
- Write a **markdown plan in the project**. Edit it yourself: delete what you do not want, mark “won’t do,” keep a “later” list.
- Optional but powerful: start from **hand-written test cases** (some founders refuse to let the model write the tests). Green tests = done. You review modularity, not every line.

### While you build

- Implement **one section of the plan at a time**. “Do section 2 only.” Test. Commit. Mark that section complete in the plan. Do not oneshot the whole product.
- **Git after every working slice.** Clean slate before each feature. Do not trust the tool’s built-in undo yet. If the agent goes on a vision quest: hard reset.
- Do not stack 5 failed prompts on dirty code. Find the fix, reset to clean, apply the clean fix once.
- Prefer **high-level end-to-end tests** (user clicks through the job) over tiny unit tests. Tests catch when the model “fixes” one thing and silently breaks another.

- Put standing instructions in Cursor rules / Windsurf rules / CLAUDE.md. Founders write hundreds of lines. It helps.
- Download API docs into a local folder and tell the agent to read them first. Live web docs are still patchy.
- Hard feature? Build a tiny standalone version first (or find a GitHub example), then say “reimplement this pattern inside the real repo.”
- Keep files small and modules clear. Stable external APIs. Giant tangled repos confuse both you and the model.
- Frameworks with strong conventions and lots of public examples (e.g. Rails) currently work better than thin-training-data languages.
- Paste screenshots of UI bugs or designs you like. Use voice if you talk faster than you type. Use the model for DNS, hosting, favicons — not just code.
- Once tests pass, refactor. Ask the model what looks repetitive.
- If stuck: paste the raw error first. Ask for 3–4 possible causes before more code. Add logging. Switch models. Cursor + Windsurf on the same project is a real tactic — different speed vs depth; pick the better output.
- IDE agent looping? Paste the same question into the model’s website UI. Sometimes that works when the IDE does not.

#### The software-factory version of this (Diana)

- You write the spec and the tests that define “done.” Agents generate implementation and iterate until tests pass. You judge output; you do not have to write most of the code.
- Some teams already keep repos with almost no hand-written code — only specs and test harnesses. That is the path to one person shipping what used to take a team.
- Give the model as much context as you would give a new employee. Context is the product.

#### Do not do this

- Prompt for the entire app in one shot and hope.
- Keep prompting on a broken tree until the file is a pile of crust.
- Skip tests because “it’s just an MVP.” That is when the model silently wrecks working parts.

**Done when:** v1 exists in git, each slice is reversible, tests cover the one core user job, and you can demo it this week — not next quarter.

## 05 Put it in front of users and iterate on purpose

Learning starts at demo, not at launch blog post. After 3–6 iterations the product is usually unrecognizable. People who watched it get better often get more committed, not less.

#### Do this

- Give the brick to hair-on-fire users immediately. Watch them use it. Do not hide behind a waitlist for six months.
- After each round: what did they try to do, where did it break, would they pay, what is still on fire?
- Recruit first users one by one. Do things that do not scale. Sit with them. Fix what blocks the core job.
- Keep the loop tight: customer need → product → deeper need → better product. Real usage beats abstract knowledge.

#### If it is not working

- You still win. You now have customer data instead of a story you told yourself.

- You often find a **better idea underneath** — the deeper structural problem, not the surface pain you started with. Going deep is how you find the actual company, especially at the edge of what models can do (the missing dev tool, the bottleneck that never cleared).
- The worst failure is not being wrong. It is never deciding — dabbling, sampling every direction, learning nothing.

**Done when:** A small set of users keeps coming back, or you have unambiguous proof the pain was fake and you can pivot with conviction instead of vibes.

## 06 Turn the company into a closed loop the AI can see

Do this after you have a real product and real users — not as a substitute for Phases 1–5. AI is not a copilot you bolt on. It should be the operating system the company runs on. Old companies are open loops: decide, do, maybe check later. Information leaks. AI-native companies capture what happened, feed it back, and improve.

### Make the whole company readable by AI

- Record meetings with an AI note-taker.
- Minimize DMs and email. Put work in channels and tools an agent can read.
- Embed agents in Slack, tickets (Linear), GitHub, customer feedback (email, Pylon), Notion/docs, sales calls, standups.
- Build dashboards for revenue, sales, engineering, hiring, ops — everything important becomes an artifact.
- Every important action should leave something the center intelligence can learn from.

### What this looks like in engineering

- An agent that can see tickets, Slack, customer feedback, GitHub, plans, sales calls, and standups can judge whether the last sprint actually helped customers — then propose the next sprint.
- Status roll-ups from managers become optional. Some teams cut sprint time in half and ship far more, because coordination stops being a human game of telephone.

When this layer exists, classic middle management (humans whose job is to route information) is mostly waste. Velocity is information flow. Every human hop you remove is a speed gain. Jack Dorsey’s point at Block: if you keep the old org chart, you miss the shift. Humans sit at the edge and steer. They do not sit in the middle and forward messages.

**Done when:** You can ask the system “what shipped, what did customers feel, what should we do next?” and get an answer from artifacts — not from chasing five people on Slack.

## 07 Keep the team tiny and the founder in the tools

This is how you staff the thing once the loops exist. Do not hire the old org chart and then “add AI.”

### Three roles, not a pyramid

- **IC (builder / operator).** Directly makes and runs things. Not only engineers. Ops, support, sales — everyone builds. People come to meetings with working prototypes, not decks.
- **DRI (directly responsible individual).** Owns one outcome: strategy + customer result. One person, one result. Not a classic manager.
- **AI founder type.** Still builds. Still coaches by example. If you are the founder, this is you. Do not delegate “our AI strategy” to a hire while you stay offline.

## Money and size

- Maximize token use, not headcount. Be willing to run an uncomfortably high API bill. It is replacing a much more expensive team.
- One person with agents can be what a pre-AI engineering team used to be. Design, HR, and admin stay lean for the same reason.
- Early-stage advantage: no legacy systems, no 1,000 people to retrain. Design AI-native from day one. Big companies often need a skunkworks team off to the side (Mutiny is the example from the talk) because changing the core while it is live is hard.
- You cannot outsource conviction. Sit with coding agents until your own assumptions about “what one person can build” break.

**Done when:** The founder still ships. Headcount stays uncomfortable. The expensive line item is model usage, not coordinators.

## The weekly loop (Phases 2–5 running together)

Phases 1 and 3 happen once (then you revisit only if you pivot). Phases 2, 4, and 5 should happen every week:

| Day focus      | What you actually do                                                                                                        |
|----------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Mon</b>     | Talk to 3–5 users or watch recordings. Write what you learned at the top of the plan markdown. Cut or add one slice.        |
| <b>Tue–Wed</b> | Build only that slice. Tests first or tests with the slice. Commit when it works. Reset if the agent thrashes.              |
| <b>Thu</b>     | Put the new slice in front of the same users. Sit with them if you can. Capture the raw errors and quotes.                  |
| <b>Fri</b>     | Update the plan: what is now true, what is later, what is dead. Refactor if tests are green. Do not start a second product. |

## Rules on the wall

1. One idea. Other boats burned.
2. Hair-on-fire users first. Everyone else can wait.
3. Ship a brick in weeks. Iterate. Do not hide for a year.
4. Plan file + tests + git slices. Never oneshot a messy repo.
5. Reset after failed agent runs. Do not layer junk.
6. 100 people who love it beat 100,000 who kind of like it.
7. Make work visible to the model. DMs are a leak.
8. Founder uses the tools. Tokens before headcount.
9. Wrong + data is better than undecided + zero learning.

This is a synthesis of public YC partner talks, not official YC advice and not a substitute for talking to your own users. Tools and models change weekly — keep the sequence, swap the tools.